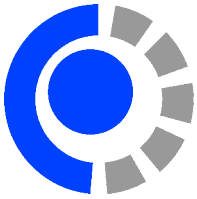



Universidad Nacional del Comahue
Facultad de Informática



Laboratorio de Compiladores e Interpretes

# Trabajo Práctico Obligatorio N°1

Franco Fabris
franco.fabris@est.fi.uncoma.edu.ar
FAI-3206

Valentina Villarroel
valentina.villarroel@est.fi.uncoma.edu.ar
FAI-3944

Neuquén Argentina

2026

# Índice

**1. Descripción del Problema** **3**

**2. Estrategias de implementación** **3**

**3. Estructuras de datos: Tabla de Símbolos y Esquema** **3**

**4. Limitaciones y Problemas del modelo** **5**

**5. Producto** **5**
5.1. Descripción del aplicativo y herramientas utilizadas . . . . . 5
5.2. Manual de uso . . . . . 5
5.3. Tabla de Errores . . . . . 6
5.3.1. Errores semánticos . . . . . 7
5.4. Ejemplos de ejecuciones exitosas . . . . . 12
5.5. Problemas hallados para el desarrollo . . . . . 13

**6. Conclusiones** **13**

**A. Instructivo de instalación** **13**

# 1. Descripción del Problema

El presente trabajo práctico tiene como objetivo extender el Analizador Sintáctico de mini-Pascal para incorporar la fase de Análisis Semántico. Como la corrección gramatical no garantiza la validez del programa, el problema central se encuentra en diseñar una Tabla de Símbolos eficiente que almacene los identificadores y sus atributos relevantes. Para lograr un acceso veloz, se requiere una estructura basada en funciones Hash, la cual debe gestionar correctamente el alcance de las variables y subprogramas mediante una pila de tablas que represente los distintos niveles de anidamiento del código fuente.

Además, se debe integrar acciones semánticas en el analizador para efectuar chequeos estáticos durante una única pasada de compilación. Estas validaciones incluyen chequeos de unicidad, para impedir la múltiple declaración o el uso de identificadores no definidos, y chequeos de tipo, que conforman el control más importante de esta fase. El aplicativo final debe asegurar la compatibilidad de operandos, la validez de las asignaciones y la correcta invocación a subprogramas (verificando cantidad y tipo de argumentos), rechazando así cualquier programa que contenga inconsistencias semánticas.

# 2. Estrategias de implementación

La implementación del analizador semántico se desarrolló en Python bajo un paradigma orientado a objetos, diseñando un sistema capaz de realizar las fases léxica, sintáctica y semántica en una única pasada de compilación. A medida que se reconocen las estructuras gramaticales del lenguaje fuente, se invoca de manera concurrente las rutinas semánticas necesarias para la validación contextual. Ante el primer error, ya sea sintáctico o semántico, la ejecución se interrumpe inmediatamente y el sistema reporta la línea exacta del código fuente donde se produjo el fallo.

Las acciones semánticas se encuentran embebidas directamente en los métodos que procesan las reglas de producción. Los chequeos de tipo (como _tipo_binario) evalúan la compatibilidad de los operandos en el momento en que se reconoce una expresión, propagando el tipo resultante hacia los nodos superiores del árbol de derivación implícito. Por ejemplo, se asegura que los operadores aritméticos solo procesen datos enteros y los relacionales devuelvan valores booleanos. En cuanto a las invocaciones de subprogramas, el analizador verifica estrictamente (mediante _chequear_argumentos) que la cantidad de argumentos provistos y sus respectivos tipos coincidan exactamente con la firma almacenada en la Tabla de Símbolos durante su declaración.

# 3. Estructuras de datos: Tabla de Símbolos y Esquema

Para la gestión de los identificadores, se diseñó la Tabla de Símbolos (TDA) utilizando diccionarios nativos de Python (dict), los cuales actúan como tablas Hash. Esto asegura que las operaciones críticas del compilador, como la declaración e inserción de símbolos (búsqueda de unicidad) y la recuperación de sus atributos, se realicen con un costo temporal constante promedio de O(1).

Cada identificador se modela mediante la clase SymbolTableEntry, la cual almacena un conjunto de atributos variables dependiendo de la naturaleza del objeto:

- Todos los símbolos poseen su lexema (nombre), categoría/atributo (Variable, Función, Procedimiento, Parámetro o Programa), tipo de dato (INTEGER, BOOLEAN o None) y la línea de declaración. También mantienen una lista de las líneas donde son referenciados (lines_referenced).
- Variables y Parámetros: Poseen un tipo de dato explícito y son los únicos evaluables en el lado derecho de una asignación o en operaciones.
- Funciones y Procedimientos: Su campo params almacena una lista ordenada con los tipos de datos de sus parámetros formales esperados. Las funciones, además, poseen un tipo de retorno y una bandera (tiene_retorno) para asegurar que se les asigne un valor dentro de su propio cuerpo.

El manejo de ámbitos se resuelve de forma dinámica mediante una pila de Tablas de Símbolos (scopeStack). El ámbito global reside en la base de la pila. Cada vez que el analizador procesa la cabecera de un subprograma,

instancia una nueva SymbolTable y la apila, registrando en ella los parámetros formales y variables locales. Al finalizar el bloque de dicho subprograma, la tabla se desapila, ocultando esos símbolos del resto del código. La resolución de referencias (_lookup) recorre esta pila desde el tope (ámbito más interno) hacia la base (ámbito global), implementando la regla del alcance más cercano.

Consideremos el siguiente fragmento de código:

```
program ejemplo;
    var x: integer;

    function sumar(a: integer): integer;
        var y: integer;
        begin
          y := 10;
          sumar := a + y + x;
        end;
    begin
      x := sumar(5);
    end.
```

Durante la compilación de este código, la estructura de la Tabla de Símbolos evolucionaría gráficamente de la siguiente manera:

1. Al procesar:

   `program ejemplo; var x: integer;`

   Se crea la Tabla Global (Base de la pila):

   - ejemplo → [Atributo: Programa, Tipo: None]
   - x → [Atributo: Variable, Tipo: INTEGER]

2. Al procesar la cabecera

   `function sumar(a: integer): integer; var y: integer;`

   Se apila una nueva tabla para el Ámbito 'sumar' (Tope de la pila):

   - a → [Atributo: Parámetro, Tipo: INTEGER]
   - y → [Atributo: Variable, Tipo: INTEGER]
     (La Tabla Global se actualiza registrando: sumar → [Atributo: Función, Tipo: INTEGER, params: [INTEGER]])

3. Durante el cuerpo de la función

   `sumar := a + y + x;`

   - El analizador busca a e y. Los encuentra inmediatamente en el tope de la pila (Ámbito 'sumar').
   - El analizador busca x. No lo encuentra en el tope, por lo que desciende a la base (Tabla Global) donde lo localiza y valida su tipo.

4. Al finalizar la función

   `end;`

   La tabla del ámbito 'sumar' se "elimina" (se desapila de scopeStack).

5. En el bloque principal

   `x := sumar(5);`

   Solo la Tabla Global está activa. La variable y o el parámetro a ya no existen en la pila, evitando colisiones de nombres o accesos indebidos desde el exterior de la función.

# 4. Limitaciones y Problemas del modelo

El diseño del analizador implementado en el archivo analizador.py presenta ciertas limitaciones arquitectónicas y funcionales derivadas de su enfoque simplificado:

- Ausencia de recuperación de errores: El mecanismo de validación interrumpe la ejecución al detectar el primer error sintáctico o semántico, lanzando una excepción ParserError que finaliza el programa inmediatamente. Al no implementar técnicas de recuperación de errores (como el modo pánico), el usuario no puede obtener un reporte completo de múltiples errores en una sola corrida, viéndose obligado a corregir y recompilar iterativamente.
- Retención de memoria en ámbitos cerrados: Aunque las tablas de símbolos locales se eliminan correctamente de la pila activa de ámbitos (self.scopeStack.pop()) al finalizar la lectura de un subprograma, los objetos de tipo SymbolTable siguen vivos en la lista self.tables. Esta retención se utiliza únicamente para imprimir el registro al final del análisis, pero en un entorno de producción o con archivos de código fuente de gran magnitud, representaría un consumo ineficiente de memoria.
- Sistema de tipos fuertemente acoplado: La validación de tipos mediante funciones como _tipo_binario tiene codificadas de forma rígida (hardcoded) las reglas para los tipos elementales INTEGER y BOOLEAN. No existe una tabla o gestor de tipos dinámico, lo que dificulta significativamente la extensibilidad del lenguaje hacia tipos de datos estructurados como arreglos (arrays), cadenas (strings) o registros (records).

# 5. Producto

## 5.1. Descripción del aplicativo y herramientas utilizadas

El analizadorSem recibe un archivo fuente escrito en mini-Pascal y realiza el análisis léxico, sintáctico y semántico. Si el programa es correcto informa `Análisis semántico exitoso` y muestra las tablas de símbolos de todos los ámbitos; si no, informa el primer error sintáctico o semántico encontrado indicando la línea del código fuente. Los errores léxicos se informan pero no detienen el análisis.

Todo el analizador esta organizado en las siguientes clases:

- `Token`: unidad léxica.
- `LexiAnalyzer`: analizador léxico, entrega un token por vez a pedido del parser.
- `SymbolTable` y `SymbolTableEntry`: tabla de símbolos de un ámbito y sus entradas.
- `Parser`: analizador sintáctico descendente recursivo predictivo con las acciones semánticas incorporadas.
- `ParserError`: excepción con la que se informan los errores sintácticos y semánticos.

Herramientas utilizadas:

- **Python 3.14** como lenguaje de implementación (se requiere la versión 3.10 o superior por el uso de `match`). No se usan bibliotecas externas.
- **PyInstaller 6.20** para generar el ejecutable `analizadorSem.exe`, que incluye el intérprete y no necesita Python instalado.

## 5.2. Manual de uso

El aplicativo se ejecuta desde la línea de comandos, teniendo como unico parametro al archivo a analizar:

```
$ analizadorSem archivodeprueba.pas
```

Si se ejecuta sin argumentos, muestra el modo de uso:

```
$ analizadorSem
Utilizar el comando: analizadorSem <archivo.pas>
```

Junto al aplicativo se entrega el directorio `pruebas/` con un archivo por cada tipo de error y dos programas correctos, que son los usados en los ejemplos de este informe.

## 5.3. Tabla de Errores

Los errores léxicos se informan por consola y el análisis continúa. Los errores sintácticos y semánticos se informan con el formato `Error sintáctico en línea N: mensaje` o `Error semántico en línea N: mensaje` y detienen el análisis. En los mensajes, `x` y `f` representan el identificador involucrado, y `T` y `U`, tipos.

Tabla 1: Errores léxicos

| N° | Mensaje | Causa | Tratamiento / corrección |
|---|---|---|---|
| 1 | `Carácter no reconocido.` | Carácter fuera del alfabeto (ej: `@`, `$`). | Se ignora el carácter y se continúa con el siguiente. |
| 2 | `Comentario no cerrado. Se esperaba una llave de cierre.` | Se llegó al fin del archivo sin encontrar la } de cierre. | Se informa y finaliza la lectura del archivo. |
| 3 | `Número mal formado. No se permiten letras después de dígitos.` | Letras inmediatamente después de un número (ej: `12ab`). | Se descarta el lexema completo y se continúa. |

Tabla 2: Errores sintácticos

| N° | Mensaje | Causa | Tratamiento / corrección |
|---|---|---|---|
| 1 | `Se esperaba 'X', se encontró 'Y'` | El token actual no es el que exige la gramática en ese punto (falta un `;`, un `then`, etc.). | Agregar o corregir el token indicado. |
| 2 | `Se esperaba 'var', 'function', 'procedure' o 'begin'` | El bloque no empieza con una sección válida. | Revisar el inicio del bloque. |
| 3 | `Se esperaba 'integer' o 'boolean'` | Tipo inexistente en una declaración. | Usar `integer` o `boolean`. |
| 4 | `Se esperaba 'function' o 'procedure'` | Se esperaba la declaración de otro subprograma. | Revisar los `;` entre subprogramas. |
| 5 | `Error en declaración de función: se esperaba '(' o ':'` | Cabecera de función mal formada. | Agregar los parámetros o el tipo de retorno. |
| 6 | `Error en declaración de procedimiento: se esperaba '(' o ';'` | Cabecera de procedimiento mal formada. | Agregar los parámetros o el `;`. |
| 7 | `Sentencia inválida` | La sentencia no empieza con un identificador, `if`, `while`, `begin`, `read` ni `write`. | Revisar la sentencia indicada. |
| 8 | `Error en expresión simple / Error en factor` | Expresión incompleta o con un elemento que no puede ser operando. | Completar o corregir la expresión. |
| 9 | `Tokens inesperados después del final del programa` | Hay código después del `end.` final. | Eliminar lo que sigue al `end.` |

Tabla 3: Errores semánticos

| N° | Mensaje | Causa | Tratamiento / corrección |
|---|---|---|---|
| 1 | `'x' ya fue declarado en este ámbito` | Dos declaraciones con el mismo nombre en un mismo ámbito (variable, parámetro, subprograma o programa). | Renombrar uno de los dos identificadores. |
| 2 | `'x' no fue declarado` | Se usa un identificador que no está en el ámbito actual ni en los exteriores. | Declararlo o corregir el nombre. |
| 3 | `operador 'OP' aplicado a T y U` | Operandos de tipo incorrecto: los aritméticos y `< <= > >=` piden enteros; `and`/`or` piden booleanos; `= <>` piden ambos del mismo tipo. | Usar operandos del tipo que exige el operador. |
| 4 | `'not' aplicado a T` | `not` sobre un operando no booleano. | Aplicarlo a una expresión booleana. |
| 5 | `signo 'OP' aplicado a T` | Signo `+`/`-` delante de un operando no entero. | Quitar el signo o usar un entero. |
| 6 | `'f' espera N argumentos, se pasaron M` | Cantidad de argumentos distinta a la de parámetros. | Ajustar la cantidad de argumentos. |
| 7 | `argumento i de 'f': se esperaba T, se pasó U` | Un argumento no coincide en tipo con su parámetro. | Pasar un argumento del tipo declarado. |
| 8 | `se asigna T a 'x' de tipo U` | Los dos lados de la asignación tienen tipos distintos. | Asignar una expresión del mismo tipo. |
| 9* | `no se puede asignar a la función 'f' fuera de su cuerpo` | Se asigna al nombre de una función fuera de ella. | Asignar el retorno solo dentro de la función. |
| 10 | `no se puede asignar a 'x'` | El destino de la asignación es un procedimiento o el programa. | Asignar solo a variables, parámetros o a la función actual. |
| 11 | `'x' no es un procedimiento` | Se usa como sentencia algo que no es un procedimiento (una función o una variable). | Usar la función dentro de una expresión. |
| 12 | `la condición del if/while debe ser BOOLEAN, es T` | La condición de un `if` o `while` no es booleana. | Usar una comparación o expresión booleana. |
| 13* | `'x' no es una función` | Una variable o parámetro usado con paréntesis. | Quitar los paréntesis. |
| 14 | `'x' no puede usarse en una expresión` | Un procedimiento o el nombre del programa usado como valor. | Llamar al procedimiento como sentencia. |
| 15 | `la función 'f' no asigna un valor de retorno` | En el cuerpo de la función nunca se asigna a su nombre. | Agregar la asignación `f := ...`. |
| 16 | `la variable de retorno 'f' no puede usarse en una expresión` | El nombre de la función actual aparece sin paréntesis en una expresión de su propio cuerpo. | Usar una variable local auxiliar. |

### 5.3.1. Errores semánticos

**Identificador redeclarado.** `sem_redeclaracion.pas`

```
1 program p;
```

```
var a: integer;
    a: boolean;
begin
  a := 1
end.
```

```
$ analizadorSem sem_redeclaracion.pas
Error semántico en línea 3: 'a' ya fue declarado en este ámbito
```

**Identificador no declarado.** sem_no_declarado.pas

```
program p;
var a: integer;
begin
  a := b + 1
end.
```

```
$ analizadorSem sem_no_declarado.pas
Error semántico en línea 4: 'b' no fue declarado
```

**Operador binario con tipos incorrectos.** sem_operador.pas

```
program p;
var a: integer;
    j: boolean;
begin
  a := a + j
end.
```

```
$ analizadorSem sem_operador.pas
Error semántico en línea 5: operador 'ADD' aplicado a INTEGER y BOOLEAN
```

**not sobre un entero.** sem_not.pas

```
program p;
var a: integer;
    j: boolean;
begin
  j := not a
end.
```

```
$ analizadorSem sem_not.pas
Error semántico en línea 5: 'not' aplicado a INTEGER
```

**Signo sobre un booleano.** sem_signo.pas

```
program p;
var a: integer;
    j: boolean;
begin
  a := -j
end.
```

```
$ analizadorSem sem_signo.pas
Error semántico en línea 5: signo 'SUB' aplicado a BOOLEAN
```

**Cantidad de argumentos incorrecta.** sem_cant_args.pas

```
program p;
var a: integer;
function doble(x: integer): integer;
begin
  doble := x * 2
end;
begin
  a := doble(a, 1)
end.
```

```
$ analizadorSem sem_cant_args.pas
Error semántico en línea 8: 'doble' espera 1 argumentos, se pasaron 2
```

**Tipo de argumento incorrecto.** sem_tipo_arg.pas

```
program p;
var a: integer;
function doble(x: integer): integer;
begin
  doble := x * 2
end;
begin
  a := doble(true)
end.
```

```
$ analizadorSem sem_tipo_arg.pas
Error semántico en línea 8: argumento 1 de 'doble': se esperaba INTEGER, se pasó BOOLEAN
```

**Asignación con tipos distintos.** sem_asignacion.pas

```
program p;
var a: integer;
    j: boolean;
begin
  j := a + 1
end.
```

```
$ analizadorSem sem_asignacion.pas
Error semántico en línea 5: se asigna INTEGER a 'j' de tipo BOOLEAN
```

**Asignación a una función fuera de su cuerpo.** sem_asig_funcion.pas

```
program p;
var a: integer;
function doble(x: integer): integer;
begin
  doble := x * 2
end;
begin
  doble := 3
end.
```

```
$ analizadorSem sem_asig_funcion.pas
Error semántico en línea 8: no se puede asignar a la función 'doble' fuera de su cuerpo
```

**Asignación a un procedimiento.** sem_asig_invalida.pas

```
1 program p;
2 var a: integer;
3 procedure muestra(x: integer);
4 begin
5   write(x)
6 end;
7 begin
8   muestra := 1
9 end.
```

```
$ analizadorSem sem_asig_invalida.pas
Error semántico en línea 8: no se puede asignar a 'muestra'
```

**Función llamada como sentencia.** sem_no_procedimiento.pas

```
1 program p;
2 var a: integer;
3 function doble(x: integer): integer;
4 begin
5   doble := x * 2
6 end;
7 begin
8   doble(a)
9 end.
```

```
$ analizadorSem sem_no_procedimiento.pas
Error semántico en línea 8: 'doble' no es un procedimiento
```

**Condición no booleana.** sem_condicion.pas

```
1 program p;
2 var a: integer;
3 begin
4   a := 10;
5   while a do
6     a := a - 1
7 end.
```

```
$ analizadorSem sem_condicion.pas
Error semántico en línea 5: la condición del while debe ser BOOLEAN, es INTEGER
```

**read sobre algo que no es una variable.** sem_read.pas

```
1 program p;
2 var a: integer;
3 function doble(x: integer): integer;
4 begin
5   doble := x * 2
6 end;
7 begin
8   read(doble)
9 end.
```

```
$ analizadorSem sem_read.pas
Error semántico en línea 8: read: 'doble' no es una variable
```

**Variable usada como función.** sem_no_funcion.pas

```
1  program p;
2  var a: integer;
3  begin
4    a := a(1)
5  end.
```

```
$ analizadorSem sem_no_funcion.pas
Error semántico en línea 4: 'a' no es una función
```

**Procedimiento usado en una expresión.** sem_en_expresion.pas

```
1  program p;
2  var a: integer;
3  procedure muestra(x: integer);
4  begin
5    write(x)
6  end;
7  begin
8    a := muestra(1) + 1
9  end.
```

```
$ analizadorSem sem_en_expresion.pas
Error semántico en línea 8: 'muestra' no puede usarse en una expresión
```

**Función sin valor de retorno.** sem_sin_retorno.pas

```
1  program p;
2  var a: integer;
3  function doble(x: integer): integer;
4  begin
5    write(x * 2)
6  end;
7  begin
8    a := doble(1)
9  end.
```

```
$ analizadorSem sem_sin_retorno.pas
Error semántico en línea 3: la función 'doble' no asigna un valor de retorno
```

**Variable de retorno usada en una expresión.** sem_var_retorno.pas

```
1  program p;
2  var a: integer;
3  function acumula(x: integer): integer;
4  begin
5    acumula := 0;
6    acumula := acumula + x
7  end;
8  begin
9    a := acumula(1)
10 end.
```

```
$ analizadorSem sem_var_retorno.pas
Error semántico en línea 6: la variable de retorno 'acumula' no puede usarse en una expresión
```

## 5.4. Ejemplos de ejecuciones exitosas

El lenguaje solo dispone de los tipos `integer` y `boolean`: no admite tipos definidos por el usuario, constantes ni estructuras como arreglos o registros. Por eso los ejemplos muestran parámetros de ambos tipos, expresiones que combinan sus operadores, y asignaciones a variables, parámetros y valores de retorno.

**Parámetros de distintos tipos, `if/else` y expresiones combinadas.** `ok_parametros.pas`

```
program parametros;
var total: integer;
    activo: boolean;

{ Recibe parámetros de distintos tipos }
function ajustar(valor: integer; duplicar: boolean): integer;
var aux: integer;
begin
  aux := valor + 1;
  if duplicar then
    ajustar := aux * 2
  else
    ajustar := aux
end;

procedure informar(n: integer; ok: boolean);
begin
  if ok and (n > 0) then
    write(n)
  else
    write(0)
end;

begin
  read(total);
  activo := total >= 10;
  total := ajustar(total, activo);
  informar(total, not activo or (total = 20))
end.
```

```
$ analizadorSem ok_parametros.pas
Análisis semántico exitoso

===== Tablas de Símbolos =====

=== Ámbito: global ===
Lexema          Tipo       Atributo       Línea decl.  Referencias
-----------------------------------------------------------------
parametros      -          programa       1            -
total           INTEGER    variable       2            25, 26, 27, 27, 28, 28
activo          BOOLEAN    variable       3            26, 27, 28
ajustar         INTEGER    función        6            11, 13, 27
informar        -          procedimiento  16           28

=== Ámbito: ajustar ===
Lexema          Tipo       Atributo       Línea decl.  Referencias
-----------------------------------------------------------------
valor           INTEGER    parámetro      6            9
duplicar        BOOLEAN    parámetro      6            10
aux             INTEGER    variable       7            9, 11, 13

=== Ámbito: informar ===
Lexema          Tipo       Atributo       Línea decl.  Referencias
-----------------------------------------------------------------
```

```
n               INTEGER     parámetro       16          18, 19
ok              BOOLEAN     parámetro       16          18
```

### 5.5. Problemas hallados para el desarrollo

- Validaciones semánticas en funciones: Fue necesario incorporar reglas de contexto específicas para asegurar que toda función asigne su valor de retorno dentro de su propio cuerpo antes de finalizar. Simultáneamente, se debió restringir cualquier intento de asignación a la función desde ámbitos externos y bloquear el uso de su variable de retorno dentro de expresiones en su propio bloque
- Gestión de ámbitos y visibilidad: Para soportar la declaración de funciones y procedimientos locales, se implementó una estructura de pila (scopeStack) que permite administrar múltiples tablas de símbolos y resolver las referencias buscando desde el entorno más interno hacia el ámbito global del programa.

## 6. Conclusiones

La integración del análisis semántico al compilador de mini-Pascal se logró exitosamente ejecutando todos los chequeos en una única pasada. Mediante el uso de diccionarios de Python como Tablas de Símbolos y una estructura de pila para el manejo dinámico de los ámbitos, se implementó un sistema eficiente capaz de almacenar identificadores, validar la unicidad, controlar la compatibilidad de tipos y verificar estrictamente la correcta invocación de subprogramas.

A pesar de cumplir con los objetivos de validación contextual, el modelo presenta limitaciones arquitectónicas derivadas de su enfoque simplificado. Entre estas restricciones destacan la interrupción abrupta de la compilación ante el primer error al carecer de rutinas de recuperación, el consumo ineficiente de memoria por la retención de ámbitos ya cerrados y la rigidez de un sistema de tipos fuertemente acoplado que dificulta su futura extensión.

## A. Instructivo de instalación

### Ejecutable (recomendado)

No requiere instalación. Se copia `analizadorSem.exe` (directorio `dist/`) a cualquier carpeta de una computadora con Windows y se ejecuta desde una consola (`cmd` o PowerShell), como se indica en el Manual de uso. No necesita Python ni otras bibliotecas.

### Desde el código fuente

En cualquier sistema operativo con Python 3.10 o superior:

```
$ python analizador.py archivodeprueba.pas
```

### Regenerar el ejecutable

```
$ pip install pyinstaller
$ pyinstaller --onefile --name analizadorSem analizador.py
```

El ejecutable queda en `dist/analizadorSem.exe`.